Stata Factor Variables

1. **Factor variable notation for categorical variables** Stata's "factor variable" notation is ideal for categorical variables and interaction terms. When Stata sees the prefix `i.` before a categorical variable that takes on k values, it expands the categorical variable into $k-1$ dummy variables. It does this temporarily for whatever command is using the factor variable. The dummies are "virtual" and are not added to your dataset.

   For example, suppose the variable `boro` takes on five values for the NYC boroughs (coded 1–5: Brooklyn, Manhattan, Staten Island, Queens, and the Bronx). Factor variable notation can be used to get means for the virtual `boro` dummy variables:

```
summ i.boro
```

   The resulting output will be:

```
Variable | Obs      Mean   Std. Dev.   Min   Max
---------+-------------------------------------
boro2    |
       2 | 437   .2448513   .4304918     0     1
       3 | 437   .1830664   .3871642     0     1
       4 | 437   .0228833   .1497028     0     1
       5 | 437   .2723112   .4456594     0     1
```

   Note that missing values in the original data carry forward as missing values in the virtual dummies.

2. **Factor variables require non-negative integer values** Factor variable notation can only be used with non-negative integer values. If `boro` were a string (e.g., K, M, R, Q, X), you could first encode it before using factor notation:

```
encode boro, gen(boro2)
```

   The result will be a numeric variable `boro2` with values 1–5. You may want to apply value labels to these numeric values if you want them to appear in your output later.

3. **Controlling the base category** By default, Stata creates $k-1$ virtual dummy variables for categorical variables. One category is excluded. In the output above, only boroughs 2–5 are shown; borough 1 was omitted. You can control which group—if any—is excluded. A few options are:

```
summ ibn.boro       // no category (or base level) excluded
summ ib2.boro       // category 2 is the excluded base level
summ ib(freq).boro  // most frequent category is the excluded base
summ ib(first).boro // first ordered category is the excluded base
```

4. **Two-way interactions with #** Factor variables are very useful for interaction terms. The # notation produces a two-way interaction between `boro` and `sex`—that is, all combinations of values that `boro` and `sex` can take on:

```
summ i.boro#i.sex
```

There are 10 possible combinations in this interaction: K-Male, K-Female, M-Male, M-Female, and so on (K is Brooklyn, M is Manhattan, etc). Again, Stata will omit one dummy variable unless you specify otherwise with `ibn.` It is not necessary to include the `i.` notation here; Stata will assume with the two-way interaction that the variables are categorical. However, I often use the notation anyway, for clarity.

5. **Interactions and main effects with ##** The ## operator produces interaction terms and main effects (that is, factorial interactions). Again, Stata will omit one dummy variable unless you specify otherwise with `ibn.`.

```
summ i.boro##i.sex
```

6. **Higher-order interactions** Factor notation can be used for higher-level interactions. For example, the following command produces a three-way interaction:

```
summ i.boro#i.sex#i.ell
```

7. **Continuous variables with `c.`** The prefix `c.` before a variable name tells Stata that the variable is continuous. This can be used to create higher-order polynomial terms and interactions between continuous and categorical variables.

   For example, the following command includes the continuous variable age and its square in the regression:

```
regress hrwage age c.age#c.age
```

   This command includes age, `female`, and their interaction in a regression:

```
regress hrwage i.female##c.age
```

8. **Other factor-variable shortcuts** There are many other things one can do with factor variables. A few examples follow. Type `help factor variables` for more ideas.

```
summ i2.boro       // uses a virtual dummy for boro==2
summ io(3 4).boro  // uses virtual dummies for boro other than 3 and 4
```

9. **Setting factor-variable defaults with `fvset`** You can use the `fvset` command to declare base settings that you intend to use repeatedly with the same variables. For example:

```
fvset base 3 varlist     // use 3 as the base for each var in varlist
fvset base none varlist  // no base category for each var in varlist
fvset base first varlist // use first category as the base (default)
fvset report [varlist]   // view current factor-variable base settings
fvset clear [varlist]    // clear current factor-variable base settings
```

10. **Factor variables and post-estimation** Aside from convenience, the real power of factor variables comes when they are combined with post-estimation commands such as `margins`.